

Building a CNN for MNIST Handwritten Digit Classification

✓ Introduction

Welcome! In this assignment, you will build a Convolutional Neural Network (CNN) to classify handwritten digits from the famous MNIST dataset. This dataset is a classic in the field of computer vision and provides a great starting point for understanding image classification with deep learning.

This notebook is structured to guide you step-by-step through the process. You will load the data, preprocess it, define a CNN model, train it, and evaluate its performance. Throughout the assignment, you will have opportunities to experiment and deepen your understanding of the concepts.

Remember to:

- **Read all instructions carefully.**
- **Execute the code cells in order.**
- **Fill in the missing code sections marked as "Students: Fill in the blanks".**
- **Answer the reflection questions in the designated Markdown cell.**
- **Experiment and explore!** Change parameters, layers, and observe the effects.

Let's get started and build our MNIST digit classifier!

Section 1: Setting Up - Imports

Before we dive into building our CNN, we need to import the necessary libraries. These libraries provide pre-built tools and functions that will make our work much easier.

Instructions:

1. **Carefully review the code cell below.** It imports libraries from TensorFlow and Keras, which are powerful frameworks for building and training neural networks.
2. **Execute the code cell by selecting it and pressing [Shift + Enter] (or the "Run" button).**
3. **Ensure there are no error messages after running the cell.** If you encounter errors, double-check that you have TensorFlow and Keras installed in your environment.

```
1 # Cell 1: Imports
2 import tensorflow as tf
3 from tensorflow import keras
```

```
4 from tensorflow.keras import layers
5 from tensorflow.keras.datasets import mnist
6 from tensorflow.keras.utils import to_categorical
```

Explanation of Imports:

- **tensorflow as tf** and **keras**: TensorFlow is the main deep learning framework, and Keras is its high-level API that simplifies building and training models. We import TensorFlow as **tf** and Keras directly for easy access to their functionalities.
- **from tensorflow.keras import layers**: This imports the **layers** module from Keras, which provides various layers for building neural networks (like convolutional layers, dense layers, etc.).
- **from tensorflow.keras.datasets import mnist**: This imports the MNIST dataset directly from Keras datasets. This is very convenient for loading and using the MNIST data.
- **from tensorflow.keras.utils import to_categorical**: This imports the **to_categorical** function, which we will use to perform one-hot encoding of our labels.

✓ Section 2: Data Loading and Preprocessing

In this section, we will load the MNIST dataset and prepare it for training our CNN model. Preprocessing steps are crucial to ensure our data is in the right format for the model to learn effectively.

Instructions:

1. **Read through the code in the cell below.** Understand how it loads the MNIST dataset and what preprocessing steps are applied.
2. **Execute the code cell.**
3. **Examine the comments in the code** to understand each preprocessing step in detail.

```
1 # Cell 2: Data Loading and Preprocessing
2 # Load the MNIST dataset
3 (x_train, y_train), (x_test, y_test) = mnist.load_data()
4
5 # Normalize pixel values to be between 0 and 1
6 x_train = x_train.astype("float32") / 255.0
7 x_test = x_test.astype("float32") / 255.0
8
```

```

9 # Add a channel dimension (for grayscale images, it's 1)
10 x_train = x_train.reshape(-1, 28, 28, 1)
11 x_test = x_test.reshape(-1, 28, 28, 1)
12
13 # One-hot encode the labels
14 num_classes = 10
15 y_train = to_categorical(y_train, num_classes)
16 y_test = to_categorical(y_test, num_classes)

```

Explanation of Data Preprocessing:

- **Loading the MNIST dataset:** `mnist.load_data()` loads the MNIST dataset, which is already split into training and testing sets (`(x_train, y_train)`, `(x_test, y_test)`). `x_train` and `x_test` contain the images (pixel data), and `y_train` and `y_test` contain the corresponding labels (digits 0-9).
- **Normalization:** `x_train = x_train.astype("float32") / 255.0` and `x_test = x_test.astype("float32") / 255.0` normalize the pixel values. Pixel values in images are typically in the range 0-255. Dividing by 255 scales them to the range 0-1. This normalization helps the neural network train faster and more effectively.
- **Adding Channel Dimension:** `x_train = x_train.reshape(-1, 28, 28, 1)` and `x_test = x_test.reshape(-1, 28, 28, 1)` reshape the data to add a channel dimension. Even though MNIST images are grayscale (single channel), CNNs in Keras expect input data to have a channel dimension. We reshape from `(number_of_images, 28, 28)` to `(number_of_images, 28, 28, 1)`. The `-1` in `reshape` means "infer the dimension based on the size of the array."
- **One-Hot Encoding:** `y_train = to_categorical(y_train, num_classes)` and `y_test = to_categorical(y_test, num_classes)` perform one-hot encoding on the labels. Instead of representing the digit '3' as a single number, one-hot encoding converts it into a vector `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`, where the 4th position (index 3) is 'hot' (value 1), and all other positions are 'cold' (value 0). This is a standard way to represent categorical labels for neural networks in multi-class classification problems. `num_classes = 10` specifies that we have 10 classes (digits 0-9).

✓ Section 3: Model Definition - Building the CNN

Now we will define the architecture of our Convolutional Neural Network (CNN). You will be building a sequential model using Keras layers.

Instructions:

1. **Carefully examine the code in the cell below.** Notice the structure of the `keras.Sequential` model.
2. **Fill in the missing parts** marked with `# Students: Fill in the blanks` to complete the model definition.
3. **Experiment!** You are encouraged to try different configurations for the layers, such as changing the number of filters in the convolutional layers, or adding more layers.

```

1 # Cell 3: Model Definition
2 # Build the CNN model.  Students: Fill in the missing parts!
3 model = keras.Sequential(
4     [
5         keras.Input(shape=(28, 28, 1)), # Input layer
6         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # Convolutional layer 1
7         layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 1
8         # Students: Add another Conv2D layer here.  Experiment with the number of filters.
9         # layers.Conv2D(____, kernel_size=(____, ____), activation="____"),
10        layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # Convolutional layer 2
11        layers.MaxPooling2D(pool_size=(2, 2)), # Max pooling layer 2
12        # Students: Add another MaxPooling2D layer here if needed.
13        # layers.MaxPooling2D(pool_size=(____, ____)), # Max pooling layer 3
14        layers.Flatten(), # Flatten layer
15        layers.Dropout(0.5), # Dropout layer
16        layers.Dense(num_classes, activation="softmax"), # Output layer
17    ]
18 )

```

Explanation of Layers:

- `keras.Input(shape=(28, 28, 1))`: This is the input layer of our model. It specifies the shape of the input images, which are 28x28 pixels with 1 channel (grayscale).
- `layers.Conv2D(32, kernel_size=(3, 3), activation="relu")`: This is a 2D Convolutional layer.
 - `32`: This is the number of filters (also called kernels). Each filter learns to detect specific features in the input image.
 - `kernel_size=(3, 3)`: This defines the size of the convolutional filter as 3x3 pixels.
 - `activation="relu"`: ReLU (Rectified Linear Unit) is the activation function. It introduces non-linearity into the model, allowing it to learn complex patterns.
- `layers.MaxPooling2D(pool_size=(2, 2))`: This is a Max Pooling layer.

- `pool_size=(2, 2)`: It reduces the spatial dimensions of the feature maps by taking the maximum value within each 2x2 window. This helps to reduce the number of parameters, control overfitting, and make the model more robust to small shifts and distortions in the input.
- `layers.Flatten()`: This layer flattens the 2D feature maps from the convolutional and pooling layers into a 1D vector. This is necessary to connect the convolutional part of the network to the fully connected (Dense) layers.
- `layers.Dropout(0.5)`: This is a Dropout layer.
 - `0.5`: This sets the dropout rate to 50%. During training, this layer randomly sets 50% of the input units to 0 at each update. This is a regularization technique that helps to prevent overfitting.
- `layers.Dense(num_classes, activation="softmax")`: This is the output Dense (fully connected) layer.
 - `num_classes`: This is set to 10 because we have 10 classes (digits 0-9).
 - `activation="softmax"`: Softmax activation ensures that the output values are probabilities, and they sum up to 1 across all classes. The output will be a vector of 10 probabilities, where each probability represents the model's confidence that the input image belongs to that specific digit class.

✓ Section 4: Model Compilation - Choosing Loss and Optimizer

Before we can train our model, we need to compile it. Compilation involves choosing an optimizer, a loss function, and metrics to evaluate the model's performance.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for the `loss` and `optimizer` parameters in `model.compile()`.
2. **Choose an appropriate loss function and optimizer** for this multi-class classification problem.
3. **In the Markdown cell after the code, explain your choices.** Why are these choices suitable for this task?

```
1 # Cell 4: Model Compilation
2 # Students: Choose an appropriate loss function and optimizer. Why did you
3 model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["
```

Explanation of Choices (To be filled by students in the reflection section):

- **Loss Function:** You need to choose a loss function that is appropriate for multi-class classification. Think about what kind of error we are trying to minimize when classifying digits into 10 categories.
- **Optimizer:** You need to choose an optimizer that will efficiently update the model's weights to minimize the loss function. Consider common optimizers used in deep learning.
- **Metrics:** We are using "accuracy" as a metric to evaluate the model's performance. Accuracy is a common metric for classification tasks, representing the percentage of correctly classified images.

✓ Section 5: Model Training - Fitting the Model to the Data

Now it's time to train our CNN model using the training data. Training involves feeding the training data to the model and adjusting its weights to minimize the loss function.

Instructions:

1. **Examine the code cell below.** You need to fill in the blanks for `batch_size` and `epochs` in `model.fit()`.
2. **Choose appropriate values for `batch_size` and `epochs`.**
3. **Run the code cell to start training.** Observe the training progress, especially the loss and accuracy on both the training and validation sets.
4. **Experiment!** Change the `batch_size` and `epochs` and see how it affects the training process and the final performance.

```
1 # Cell 5: Model Training
2 # Students: Adjust the batch size and number of epochs. What happens if yc
3 model.fit(x_train, y_train, batch_size=128, epochs=15, validation_split=0.1
```

Explanation of Training Parameters:

- **`batch_size`:** This determines the number of training samples processed in each mini-batch during training. A larger batch size can speed up training but might require more memory. A smaller batch size can lead to more noisy updates but might generalize better.
- **`epochs`:** One epoch represents one complete pass through the entire training dataset. More epochs can potentially lead to better training but also increase the risk of overfitting, where the model learns the training data too well and performs poorly on unseen data.

- **validation_split=0.1**: This reserves 10% of the training data as a validation set. During training, the model's performance is evaluated on this validation set after each epoch. This helps to monitor for overfitting and tune hyperparameters.

Section 6: Model Evaluation - Assessing Performance on Test Data

After training, we need to evaluate our model's performance on the test dataset. This gives us an estimate of how well the model generalizes to unseen data.

Instructions:

1. **Run the code cell below.**
2. **Observe the output.** It will print the test loss and test accuracy.
3. **Think about the results.** Is the test accuracy satisfactory? How does it compare to the training and validation accuracy you observed during training?

```
1 # Cell 6: Model Evaluation
2 loss, accuracy = model.evaluate(x_test, y_test, verbose=0)
3 print(f"Test loss: {loss:.4f}")
4 print(f"Test accuracy: {accuracy:.4f}")
```

Section 7: Reflection and Answers to Questions

This is an important section! Take some time to reflect on what you have learned and answer the following questions in detail. Your thoughtful answers will demonstrate your understanding of the concepts covered in this assignment.

Reflection Questions:

1. **Conv2D Layer:** What is the role of the Conv2D layer? How do the **kernel_size** and the number of filters affect the learning process? *Hint: Experiment by changing these values in Cell 3.*
2. **MaxPooling2D Layer:** What is the purpose of the MaxPooling2D layer? How does it contribute to the model's performance? *Hint: Try removing or adding a MaxPooling2D layer and see what happens.*
3. **One-Hot Encoding:** Why do we use one-hot encoding for the labels?
4. **Flatten Layer:** Why do we need the Flatten layer before the Dense layer?

5. **Optimizer and Loss Function:** What optimizer and loss function did you choose in Cell 4? Explain your choices. Why is categorical cross-entropy a suitable loss function for this task? Why is Adam a good choice of optimiser?
6. **Batch Size and Epochs:** How did you choose the batch size and number of epochs in Cell 5? What are the effects of changing these parameters? *Hint: Experiment!*
7. **Dropout:** Why is the Dropout layer included in the model?
8. **Model Architecture:** Describe the overall architecture of your CNN. How many convolutional layers did you use? How many max pooling layers? What is the final dense layer doing?
9. **Performance:** What accuracy did you achieve on the test set? Are you happy with the result? Why or why not? If you're not happy, what could you try to improve the performance?

Tips and Explanations:

- **Normalization:** Dividing the pixel values by 255 normalizes them to the range [0, 1]. This is important for training neural networks.
- **Reshaping:** The `reshape` operation adds a channel dimension to the images. For grayscale images, the channel dimension is 1.
- **One-Hot Encoding:** `to_categorical` converts the class labels (0-9) into one-hot encoded vectors.
- **Conv2D Parameters:** The `kernel_size` determines the size of the convolutional filter (e.g., 3x3). The number of filters determines how many different features are learned.
- **MaxPooling2D Parameters:** The `pool_size` determines the size of the pooling window (e.g., 2x2).
- **Optimizer:** The optimizer is the algorithm used to update the model's weights during training.
- **Loss Function:** The loss function measures the error between the model's predictions and the true labels.
- **Batch Size:** The batch size is the number of samples processed in each training iteration.
- **Epochs:** An epoch is one complete pass through the entire training dataset.
- **Dropout:** Dropout is a regularization technique that helps prevent overfitting.

Remember to run each cell to see its output. Experiment with the code and try to understand how different parameters affect the model's performance. Good luck! """"

✓ Conclusion and Submission

Congratulations on completing this notebook assignment! You have successfully built and trained a Convolutional Neural Network to classify handwritten digits from the MNIST dataset. You've explored key concepts like convolutional layers, pooling layers, activation functions, optimizers, loss functions, and training procedures. To further solidify your understanding, consider the following:

- **Review your notebook:** Go back through each section, reread the explanations, and make sure you understand the code and the concepts.
- **Experiment further:** Try different CNN architectures, add more layers, change hyperparameters, and see how it affects the performance. Explore other optimizers or loss functions.
- **Reflect on your learning:** Think about the challenges you faced and how you overcame them. What were the most important takeaways for you from this assignment?

Submission Instructions

To submit your assignment:

1. **Save your notebook:** Ensure all your work, including code cells, outputs, and answers to reflection questions, is saved in the notebook.
2. **Print the notebook as a .pdf file** and submit it to Canvas.

Deadline: February, 12th

